

GUIA DE REQUISITOS TÉCNICOS

SPRINT 4 — MOTIVA

Migração para Spring Boot + JPA + API REST

OBJETIVO GERAL Transformar o sistema MOTIVA — hoje um protótipo de console com persistência via JDBC puro (Sprint 3) — em uma API REST profissional usando Spring Boot e Spring Data JPA, reaproveitando o banco Oracle e as tabelas que vocês mesmos modelaram na Sprint 3.

1. Contexto

Esta sprint é a grande virada tecnológica do Challenge. O sistema que vocês construíram até aqui:

Sprint	O que foi construído
Sprint 1	Domínio base: TrechoRodovia, encapsulamento e protótipo de console.
Sprint 2	Motor de regras: classes abstratas (IntervencaoOperacional), interfaces (MonitoravelViaIoT) e polimorfismo.
Sprint 3	Persistência: JDBC puro, pattern DAO, CRUD completo no Oracle e relatório com histórico.
Sprint 4	Framework: Spring Boot + JPA + REST.

A mensagem pedagógica desta sprint é exatamente a da Aula 13

```
// Sprint 3 (JDBC puro): ~40 linhas por operação
Connection conn = DriverManager.getConnection(...);
PreparedStatement stmt = conn.prepareStatement("INSERT INTO ...");
// ... parâmetros, execute, fechar tudo no finally ...
```

```
// Sprint 4 (Spring Data JPA): 1 linha
repository.save(trecho);
```

IMPORTANTE Assim como na Sprint 3, use os nomes e a modelagem do SEU projeto. Se sua classe se chama MotorPrioridade em vez de GeradorRelatorio, ou sua tabela tem campos diferentes, adapte. O que não pode mudar são os conceitos exigidos nos requisitos abaixo.

2. A grande virada: DAO → camadas Spring

Sprint 3 (JDBC puro)	Sprint 4 (Spring Boot)	Papel
model/ + record interno do DAO	@Entity em model/	Classe mapeada para a tabela via JPA.
EquipeManutencaoDAO (375 linhas)	EquipeManutencaoRepository (interface, ~5 linhas)	Acesso a dados sem SQL manual.
ConexaoBD (Singleton)	application.properties + pool do Spring	Conexão gerenciada pelo framework.
service/GeradorRelatorio	@Service	Regras de negócio (continua existindo!).
Main.main() com System.out.println	@RestController + JSON	Interface do sistema vira HTTP.
Teste manual no console	Testes com cURL / Postman	A API passa a ser consumida por qualquer cliente.

CONCEITO-CHAVE Repare: o pattern DAO não morre — ele é absorvido. O Repository do Spring Data é um DAO gerado pelo framework em tempo de execução.

3. Requisitos funcionais

3.1 Arquitetura em camadas (obrigatória)

```
src/main/java/br/com/motiva/
+-- MotivaApplication.java # @SpringBootApplication (main)
+-- model/                 # @Entity — suas entidades da Sprint 3
+-- repository/            # Interfaces JpaRepository — SEM SQL
+-- service/               # @Service — regras de negócio (motor de prioridade)
+-- controller/            # @RestController — endpoints REST
```

FLUXO ENTRE CAMADAS Cada camada só conversa com a vizinha: Controller → Service → Repository → Banco.

3.2 CRUD REST completo

Para cada entidade principal (no mínimo: Trecho, Equipe e Intervenção), expor:

Método HTTP	Rota	Ação	Resposta de sucesso / erro
GET	/api/trechos	Listar todos	200 OK
GET	/api/trechos/{id}	Buscar por ID	200 OK / 404 Not Found se não existir
POST	/api/trechos	Criar	201 Created com o corpo criado
PUT	/api/trechos/{id}	Atualizar	200 OK / 404 Not Found
DELETE	/api/trechos/{id}	Remover	204 No Content / 404 Not Found

Códigos HTTP corretos fazem parte da nota (Aula 16).

3.3 Relatório de prioridade como serviço REST

O motor da Sprint 2/3 vira endpoint:

Método	Rota	Ação
POST	/api/relatorios	Gera o relatório a partir dos trechos atuais e persiste no histórico.
GET	/api/relatorios	Lista o histórico.
GET	/api/relatorios/periodo?inicio=...&fim=...	Consulta históricos por período.

REGRA DE NEGÓCIO A lógica de classificação (URGENTE = 80cm, CRÍTICO = 50cm, ATENÇÃO = 25cm, NORMAL < 25cm — ou os seus limiares da Sprint 2) deve viver no Service, nunca no Controller.

3.4 Queries Spring Data

Cada Repository deve demonstrar derived queries (o Spring gera o SQL pelo nome do método):

```
List findByNivelVegetacaoCmGreaterThanOrEqualTo(Double minimo);
List findByTipo(String tipo);
```

4. Requisitos técnicos

Item	Exigência
Framework	Spring Boot 4.1.x (conforme o Initializr da Aula 13).
Java	17.
Build	Maven (pom.xml).
Banco	Oracle FIAP: oracle.fiap.com.br:1521:ORCL, reaproveitando as tabelas da Sprint 3.
Driver	com.oracle.database.jdbc:ojdbc17 (dependência Maven; sem JAR manual).
Starters	spring-boot-starter-web e spring-boot-starter-data-jpa.
Configuração	application.properties. Nunca coloque credenciais em código Java.

É proibido nesta sprint

- Usar Connection, PreparedStatement ou ResultSet (isso era a Sprint 3).
- Escrever SQL manual dentro de Services ou Controllers.
- Colocar lógica de negócio dentro de @RestController.
- Usar System.out.println como interface do sistema; a interface agora é o JSON.

5. Entregáveis

5.1 Código-fonte

- Projeto Spring Boot + Maven funcionando com `mvn spring-boot:run`.
- Entities JPA para as entidades do SEU projeto (com `@Entity`, `@Table`, `@Id`, `@GeneratedValue` com `@SequenceGenerator` ligado às SUAS sequences da Sprint 3).
- Repositories como interfaces `JpaRepository`, incluindo derived queries.
- Services com regras de negócio: validações da Sprint 1 e motor de prioridade da Sprint 2.
- Controllers REST com CRUD completo e códigos HTTP corretos.
- Endpoints de relatório: gerar, consultar histórico e filtrar por período.

5.2 Documentação

- README.md com instruções de execução, tabela de endpoints documentada e exemplos cURL.
- Print ou registro de pelo menos 6 requisições funcionando, incluindo CRUD e relatório.

5.3 Git

- Repositório GitHub com commits incrementais; não deixar todos os commits para a véspera.
- `.gitignore` correto, incluindo `target/` e arquivos da IDE.
- Não incluir senhas reais no repositório; usar placeholders ou variáveis de ambiente.

6. Critérios de avaliação

Critério	Peso
Migração JPA correta: Entities + Repositories, zero JDBC manual.	30%
API REST com CRUD completo e códigos HTTP corretos (200/201/204/404/400).	30%
Camada @Service com regras de negócio: motor de prioridade + validações.	20%
Endpoints de relatório (gerar, histórico e período) com persistência.	10%
README com endpoints documentados, exemplos cURL e boas práticas Git.	10%

Bônus (até +1,0 ponto)

Implementação adicional	Bônus
DTOs para separar a entidade do contrato da API.	+0,3
Bean Validation (@Valid, @NotNull, @Positive).	+0,3
@ControllerAdvice para tratamento global de erros.	+0,2
Teste de API com @SpringBootTest + MockMvc (Aula 15).	+0,2

7. Conceitos em foco: perguntas de reflexão

Responda no README da entrega — estas perguntas valem a rubrica de “Documentação”:

1. Por que o Repository é uma interface e não uma classe? Quem escreve a implementação e quando?
2. O pattern DAO da Sprint 3 “morreu” na migração ou só mudou de forma? Explique com as suas palavras.
3. Por que a validação de nivelVegetacao >= 0 (Sprint 1) deve ficar no Service e não no Controller?
4. No JDBC puro vocês escreviam SQL. Onde está o SQL do findByTipo()? Quem o gerou?

8. Referências

- Aula 13 — Spring Boot + JPA: configuração, @Entity, Repository, Service, Controller e cURL.
- Spring Initializr (já configurado): start.spring.io
- Spring Data JPA: <https://docs.spring.io/spring-data/jpa/reference/>

BOA SPRINT!